



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2133 – Estructuras de Datos y Algoritmos

2026 - 1

Taller 4

Objetivos

- Diseñar e implementar estructuras de datos basados en árboles AVL.
- Comprender como mantener un árbol balanceado mediante rotaciones.
- Utilizar árboles AVL para realizar consultas eficientes sobre intervalos de valores.

Introducción

En lo profundo de un bosque aparentemente tranquilo, algo extraño está ocurriendo. Lo que parecían simples árboles... en realidad son Sudowoodo. Decenas de ellos han comenzado a agruparse, imitando el entorno y bloqueando caminos, confundiendo a entrenadores y dificultando cualquier tipo de exploración.



el Profesor Oak ha descubierto que cada Sudowoodo posee un identificador único, y que organizarlos correctamente es clave para entender su comportamiento. El problema es que estos Pokémon siguen apareciendo sin orden alguno, y distinguirlos manualmente se ha vuelto prácticamente imposible.

Tu misión será implementar una estructura basada en árboles AVL que permita registrar a cada Sudowoodo y consultar rápidamente cuáles cumplen ciertas condiciones.

¡Invasion de Sudowoodo!



Problema

En la ciudad, los caminos han sido bloqueados por una gran cantidad de Sudowoodo, Pokemon que imitan arboles y dificultan distinguirlos del entorno.

El Profesor Oak ha decidido registrar a cada Sudowoodo mediante un identificador unico. Sin embargo, debido a la gran cantidad de datos, es necesario utilizar una estructura eficiente que mantenga la informacion ordenada en todo momento.

Debido a la constante aparición de Sudowoodo, no es posible definir un límite máximo de registros. Por esta razón, se requiere una estructura dinámica que permita manejar una cantidad arbitraria de datos.

Para esto, se utilizara un arbol AVL, el cual permite insertar elementos manteniendo el balance del arbol.

Tu mision sera trabajar con esta estructura, construyendo el arbol y realizando operaciones sobre el.

Entidades

Sudowoodo: Cada Sudowoodo cuenta con un ID único. todos los valores son numericos.

Flujo

Para esta parte, contarás con una estructura de arbol AVL parcialmente implementada.

Primero recibirás un entero N que indica la cantidad de eventos a procesar.

Luego recibirás N eventos, cada uno correspondiente a una operación sobre el árbol AVL. Los eventos deberán procesarse en el orden en que aparecen.

Parte Formativa - Eventos

ENTER {X}

En este evento, un nuevo Sudowoodo con ID **X** aparece en el bosque y debe ser registrado en el árbol AVL.

Tu tarea será implementar en C la función de inserción del árbol AVL utilizando **recursividad**. Para esto, deberás seguir los siguientes pasos:

1. Insertar el nuevo ID siguiendo la lógica de un árbol binario de búsqueda de forma **recursiva**:
 - Si el nodo actual es NULL, crear un nuevo nodo.
 - Si **X** es menor que el nodo actual, insertar en el subárbol izquierdo.
 - Si **X** es mayor, insertar en el subárbol derecho.
 - Si **X** ya existe, no se debe insertar nuevamente.
2. Al retornar de la recursión, actualizar la altura del nodo actual con `height()`.
3. Calcular el **factor de balance** del nodo con `getbalance()`.
4. Si el nodo se desbalancea (es decir, el valor absoluto del balance es mayor a 1), aplicar una de las siguientes rotaciones:
 - Caso LL: rotación simple a la derecha.
 - Caso RR: rotación simple a la izquierda.
 - Caso LR: rotación izquierda sobre el hijo izquierdo y luego rotación derecha.
 - Caso RL: rotación derecha sobre el hijo derecho y luego rotación izquierda.
5. Retornar el nodo actual luego de aplicar las modificaciones necesarias. Este valor será utilizado por la llamada recursiva anterior.

El resultado esperado debe mostrar el siguiente output:

output:
1 Pokemon con ID {X} insertado

FIND {X}

En este evento deberás determinar si el Sudowoodo con ID **X** se encuentra registrado en el árbol AVL.

Si el valor existe en el árbol, deberás indicarlo explícitamente en el output. En caso contrario, deberás indicar que dicho Sudowoodo no se encuentra registrado.

Este evento te permitirá practicar la búsqueda sobre un árbol AVL, aprovechando la propiedad de orden que mantiene esta estructura.

El resultado esperado debe mostrar el siguiente output:

output:
1 Sudowoodo con ID {X} encontrado

o bien

output:
1 Sudowoodo con ID {X} no encontrado

(PROPUESTO) DEPTH

En este evento deberás retornar la altura actual del árbol AVL.

La altura del árbol corresponde a la longitud del camino más largo desde la raíz hasta una hoja. Este evento te permitirá analizar cómo cambia la estructura del árbol a medida que se insertan nuevos Sudowoodo y cómo el balance del AVL mantiene su altura acotada. **OJO:** un árbol AVL no necesariamente implica que todas sus caminos sean de la misma longitud

El resultado esperado debe mostrar el siguiente output:

output:	
1	Profundidad actual del arbol: {D}

(PROPUESTO) ORDER

En este evento deberás imprimir todos los Sudowoodo registrados actualmente en el árbol AVL en orden creciente según su ID.

Para lograrlo, deberás realizar un recorrido **inorder** sobre el árbol. Este recorrido visita primero el subárbol izquierdo, luego el nodo actual y finalmente el subárbol derecho. Debido a la propiedad de orden de los árboles binarios de búsqueda, esto permite obtener los IDs de menor a mayor.

Tu tarea será implementar una función recursiva que recorra el árbol en este orden e imprima cada ID encontrado.

El resultado esperado debe mostrar el siguiente output:

output:	
1	Sudowoodos en orden:
2	{ID_1}
3	{ID_2}
4	{ID_3}
5	...
6	{ID_N}

Parte Evaluada - Grupo 2

Para estos ejercicios, cada nodo del árbol AVL contará con un atributo adicional `size`, que representa la cantidad de nodos presentes en el subárbol cuya raíz es dicho nodo. Es decir:

$$\text{size} = 1 + \text{node} \rightarrow \text{left} \rightarrow \text{size} + \text{node} \rightarrow \text{right} \rightarrow \text{size}$$

Puedes asumir que cuentas con una función `size(Node * node)` que retorna el `size` de un nodo ya implementada, solo debes preocuparte por usarla.

a) Implemente el evento FIND-K {X}

La función `find-k(x)` entrega la posición del nodo con ID `x` en una lista de ID ordenados por clave de menor a mayor, donde la menor está en posición 1 y la mayor N

Por ejemplo, si el árbol AVL tiene almacenados los ID

2 7 8 10

`find-k(7)` entrega 2 y `find-k(8)` entrega 3.

Tu tarea será implementar en C la función `find-k(x)` recursiva que encuentre la posición del nodo con ID `x`.

La definición de la función es la siguiente:

```
1 int find_k(Node* node, int position, int x);
```

Donde:

- `node` es el nodo actual que se está revisando.
- `position` es una variable auxiliar que irá almacenando la posición.
- `x` es el ID del nodo cuya posición se debe encontrar
- El llamado inicial a la función es: `find_k(root, 0, X)`

El algoritmo de `find-k(x)` es:

1. Si el nodo actual (`node`) es `NULL`, retorna 0.
2. Si `x` es menor que el valor del nodo actual (`node`), continúa la búsqueda en el subárbol izquierdo sin modificar la variable de posición (`position`).
3. Si `x` es mayor que el valor del nodo actual, lo que ocurre es que todos los nodos del hijo izquierdo y además el nodo actual, son menores que el nodo `x`, por lo tanto, debemos sumar todos esos nodos a la variable `position`, de esta forma, vamos a ir acumulando cuantos nodos son menores en el arreglo ordenado que el nodo `X`. Por lo tanto, continuamos la búsqueda en el subárbol derecho pero actualizamos la variable `position` de esta forma:

$$\text{position} = \text{position} + \text{size}(\text{node} \rightarrow \text{left}) + 1$$

4. Si `x` es igual al valor del nodo actual, la posición buscada corresponde a todos los nodos que hemos venido acumulando (lo que está guardado en la variable `position`) y además, le sumamos todos los nodos del subárbol izquierdo del nodo actual, ya que estos también son menores que `X`, por lo tanto retornamos:

$$\text{position} + \text{size}(\text{node} \rightarrow \text{left}) + 1$$

Puedes asumir que el ID `X` existe en el árbol.

El resultado esperado debe mostrar el siguiente output:

output:

```
1 | Sudowoodo con ID {X} esta en la posicion {K}
```

Nota: Ten en cuenta que la complejidad de este algoritmo es $\mathcal{O}(\log N)$ siendo N la cantidad de nodos en el árbol, ya que solo hacemos una búsqueda típica pero actualizando un valor acumulado.

b) Diseña la función FIND-VALUE {K}

Dado un entero k , tu tarea será encontrar el nodo cuyo ID se encuentra en la posición k del arreglo ordenado.

Por ejemplo, si los valores almacenados en el árbol fueran:

2 7 8 10

entonces:

- Para $K = 1$, se debería retornar el Pokémon con ID 2.
- Para $K = 2$, se debería retornar el Pokémon con ID 7.
- Para $K = 3$, se debería retornar el Pokémon con ID 8.

La posición debe contarse desde 1.

La definición de la función es:

```
1 Node * find_value(Node * node, int k)
```

Donde **node** es el nodo actual que estamos revisando y k es la posición actual del nodo que estamos buscando.

Este ejercicio es lo opuesto al anterior, antes recibíamos un nodo X y debíamos retornar la posición K , ahora recibes la posición K y debes retornar el nodo X .

Explique y escriba el pseudocódigo de la función con complejidad $O(\log(N))$ utilizando las propiedades del árbol AVL.

Puedes asumir que K siempre será válido.

Hint: Tenga en cuenta la implementación realizada para el evento anterior. Plantearse las siguientes preguntas les será útil para diseñar su algoritmo:

- En la función anterior, la variable position iba aumentando al bajar por la recursión, en este algoritmo ¿la variable k debe aumentar o disminuir al bajar por la recursión?
- En la función anterior, la variable position se actualizaba al bajar por la recursión por el lado derecho, quedaba igual cuando se bajaba por el lado izquierdo, en este algoritmo ¿la variable k debe cambiar al bajar por la izquierda o por la derecha?
- ¿Al modificar el valor de k , como debo hacerlo? ¿Que información queremos tener almacenada al modificar el k tal que nos permita encontrar el nodo que queremos?

No debe responder estas preguntas en su solución, solo son una guía que les podrían ser útiles.

BONUS Implementar el evento FIND-VALUE K

Dada su solución anterior, implementelo en C.

El resultado esperado debe mostrar el siguiente output:

output:

```
1 | Sudowoodo en la posicion {K}: {ID}
```

Input

Importante: En este taller trabajarás sobre un árbol AVL, por lo que el flujo del programa estará basado en operaciones sobre esta estructura.

El archivo de input comenzará con un número **N** que corresponde a la cantidad de eventos a recibir. Luego, se entregarán los **N eventos**, cada uno con la información necesaria para realizar una operación sobre el árbol AVL.

Los eventos posibles corresponden a inserciones, búsquedas, recorridos en orden y consultas sobre la altura del árbol.

Como ejemplo tenemos el siguiente input:

input	
1	9 # Numero de eventos a recibir
2	ENTER 15
3	ENTER 7
4	ENTER 20
5	ENTER 9
6	ENTER 25
7	FIND 12
8	FIND-LCA 15 25
9	FIND-DIST 7 15 1
10	DEPTH

output	
1	Pokemon con ID 15 insertado
2	Pokemon con ID 7 insertado
3	Pokemon con ID 20 insertado
4	Pokemon con ID 9 insertado
5	Pokemon con ID 25 insertado
6	Pokemon con ID 12 no encontrado
7	El ancestro comun mas bajo entre 15 y 25 es: 15
8	Distancia entre 7 y 15: 1
9	Profundidad actual del arbol: 3

Evaluación (Solo parte evaluada)

La nota de tu taller es calculada a partir de testcases privados de input/output que se evaluarán cada uno en forma binaria, es decir, 1 punto por output correcto y 0 puntos por output incorrecto.

Se entregarán testcase publicos para que puedas probar tu implementación durante el taller.

Ademas para la parte teorica del evaluado, se espera una respuesta en sus propias palabras que se evaluara de manera manual. Por lo que se espera que el estudiante pueda explicar con sus propias palabras ya sea en pseudocodigo o en un parrafo como resolveria el problema.

Finalmente el Bonus tambien se evaluara con test cases.

Integridad académica e IA

Cualquier acto deshonesto o fraude académico esta estrictamente prohibido. Se prohíbe el uso de herramientas de inteligencia artificial y el uso o manipulación de otros dispositivos electrónicos como laptops, tablet, teléfonos durante la evaluación.

Proceso de Entrega

Conexión a internet

Una vez iniciado sesión en el escritorio, seleccionar el icono de redes arriba a la derecha. "*availablenetwork*" → "*eduroam*". Para conectarse a eduroam, deben tener la siguiente configuración:

- Tener seleccionado la opción "No CA certificate is required".
- En la opción de "*Inner authentication*" seleccionar "*PAP*".
- Luego debe ingresar tus credenciales usuario uc. **IMPORTANTE:** en el usuario debes incluir el @, no solo el nombre.
- Ejemplos correctos: "*nombre@uc.cl*"; "*nombre@estudiante.uc.cl*", Incorrecto: "nombre" sin el @.

Descargar taller

Para descargar el código base que con el que trabajaran, deben abrir una terminal, estar conectados a internet y ejecutar `descargar-taller nombre_archivo.zip`

Donde el *nombre_archivo* debe ser reemplazado con el nombre que se avisara al inicio de cada taller.

Editor de código

El editor de código de la imagen se llama "*lite – xl*". Para abrir su trabajo tienen varias alternativas, las recomendables son:

- Abrir el editor de código, buscar la sección de "*file*" y seleccionar "*Open*" y buscar la carpeta donde quieres trabajar.
- Ir al explorador de archivos, buscar la carpeta donde quieres trabajar, hacer click derecho en la carpeta y seleccionar "*Openwith...*" y seleccionar el editor.
- **IMPORTANTE:** El editor no tiene autoguardado, por lo que cada vez que hagan cambios deben usar "*CTRL + S*" para guardar. Si ustedes no guardan sus archivos e intentan compilar, el proceso de compilado no guarda automáticamente, por lo que su código se compilara con la ultima versión que hayan guardado, **DEBEN GUARDAR ANTES DE COMPILAR.**

Espacio de trabajo

Al abrir el explorador de archivos o una terminal, verán una carpeta llamada código. Es **FUNDAMENTAL** que todo el trabajo que hagan lo tengan dentro de esa carpeta. Eso les asegura que en caso de que la imagen falle y tengan que reiniciar el computador, todo lo que está guardado en esa carpeta se mantendrá.

Uso del entorno de trabajo

En su carpeta de trabajo tendrán al menos los siguientes elementos:

- src: carpeta con su código
- makefile: archivo para compilar
- tests: carpetas de tests publicas
- run.sh: script para ejecutar todos los tests

Para compilar, en una terminal usen:

```
make
```

Para usar el script que ejecuta todos los tests, usan:

```
./run.sh
```

El run.sh genera una carpeta llamada outputs que guarda para cada test, el output de su programa.

Si desean ejecutar un único test ocupan

```
./ejecutable ruta_al_input_del_test.txt archivo_output.txt
```

Donde ejecutable es el nombre del archivo que se genera al compilar, puedes revisarlo en las carpetas dentro de src, ejemplo de ejecutables: "pokemones", "magnemites". Ejemplo de ruta al archivo de input: tests/F1/pokemones/easy-0/input.txt y output.txt es un archivo txt que crean y es donde ira su salida.

En caso de querer debugear algún error de un test en especifico usar **valgrind**

```
valgrind ./ejecutable ruta_a_input.txt output.txt
```

Entrega Taller

Para entregar, también es importante que estén conectados a internet, entrar en una terminal, posicionarse en la ruta donde esta la carpeta de su entrega (no dentro de la carpeta misma). Finalmente, ejecutar

```
entregar-taller nombre_carpeta TX nAlumno1_nAlumno2_...
```

Donde: "*nombre_carpeta*" es el nombre de la carpeta que quieren entregar, por ejemplo: "*T2_evaluado_G2*". "*TX*" donde X corresponde al taller que se esta entregando: por ejemplo: "T2". Y *nAlumno1_nAlumno2_...* que son los números de alumnos de los integrantes del grupo que trabajaron separados por "_" (Si están realizando este taller de manera solitaria, solamente escriben su numero de alumno).
Ejemplo Correcto:

```
12345678_23456789 20387645 20387645_12345678_23456789
```

Ejemplo Incorrecto:

```
1234567823456789 Nombre1_12345678 Nombre1_Nombre2_Nombre3
```